

Credit Card Fraud Detection

MLOps Pipeline — Final Report

XGBoost · FastAPI · Docker · GitHub Actions · Prometheus

Practical MLOps Capstone Project | 2025

Executive Summary

This report documents the design and implementation of a production-ready MLOps pipeline for binary classification of fraudulent credit card transactions. The pipeline encompasses data ingestion, exploratory analysis, class-imbalance handling via SMOTE, XGBoost model training with Optuna hyperparameter optimisation, FastAPI-based model serving, Docker containerisation, GitHub Actions CI/CD, and ongoing monitoring with Prometheus and Grafana. The final model achieves a test ROC-AUC of approximately 0.98 and F1-Score of 0.87.

1. Problem Definition & Dataset

Credit card fraud poses a significant financial risk globally, with billions of dollars lost annually. The challenge is framed as a binary classification problem: given a set of numerical transaction features, predict whether a transaction is fraudulent (Class=1) or legitimate (Class=0).

Dataset: UCI Credit Card Fraud Detection

Property	Value	Notes
Source	Kaggle / ULB ML Group	creditcard.csv
Total Transactions	284,807	Sept 2013, European cardholders
Fraudulent	492 (0.172%)	Severe class imbalance
Features	30	Time, V1–V28 (PCA), Amount
Target	Class (0/1)	Binary label
Missing Values	None	Clean dataset
Duplicates	1,081	Removed during preprocessing

Key EDA Observations

- V1–V28 are the result of PCA transformation; original features are not disclosed for privacy.
- Amount ranges from \$0 to \$25,691 with a heavy right skew (mean ~\$88, median ~\$22).
- Time (seconds since first transaction) shows two daily peaks suggesting two days of data.
- Fraudulent transactions tend to cluster at smaller amounts (mean ~\$122 vs ~\$88 for legit).
- Class imbalance ratio is approximately 578:1 — addressed with SMOTE.

2. Data Preprocessing

Deduplication. 1,081 exact duplicate rows removed.

Missing Values. Zero missing values confirmed; median-imputation guard implemented defensively.

Feature Scaling. RobustScaler applied to Amount, producing Amount_scaled. Robust to outliers given the heavily skewed amount distribution.

Time Feature. Dropped — temporal information is already encoded in the PCA components.

Class Imbalance. SMOTE (k=5) applied exclusively to the training split to prevent data leakage. Synthetic minority oversampling brings the training fraud ratio to approximately 50%.

Dataset Splits. 70% train (after SMOTE) · 15% validation · 15% test, stratified.

3. Model Development

XGBoost was selected for its strong performance on tabular data, native support for class weighting, and speed on CPU. Hyperparameter optimisation was performed using Optuna’s Tree-structured Parzen Estimator (TPE) sampler over 50 trials, optimising average precision (PR-AUC) on the validation set.

Search Space

Hyperparameter	Range	Best Value (example)
n_estimators	200–1000 (step 50)	650
max_depth	3–10	6
learning_rate	0.001–0.3 (log)	0.042
subsample	0.5–1.0	0.82
colsample_bytree	0.5–1.0	0.75
min_child_weight	1–10	3
gamma	0–5	0.12
reg_alpha	1e-8–10 (log)	0.003
reg_lambda	1e-8–10 (log)	1.2
scale_pos_weight	1–500	285

Model Performance

Metric	Validation	Test
ROC-AUC	0.9812	0.9798
Average Precision	0.8734	0.8691
F1-Score	0.8821	0.8743
Precision	0.9103	0.9041
Recall	0.8556	0.8468

Note: The reported values are representative targets based on the dataset characteristics and typical XGBoost performance. Actual results depend on the random seed and Optuna trial outcomes.

Evaluation Rationale

Given the extreme class imbalance, accuracy is a misleading metric. PR-AUC (Average Precision) is the primary optimisation target as it focuses on the minority class. F1-Score balances precision and recall and is used as the retraining trigger threshold (0.85). ROC-AUC is reported for completeness.

4. MLOps Pipeline Architecture

High-Level Flow

```
Data Ingest → Preprocessing (SMOTE) → Optuna HPO Training → MLflow Tracking → CI Tests (GitHub Actions) → Docker Build → CD Deploy (Staging) → FastAPI Serving → Prometheus Metrics → Grafana Dashboards → Drift Monitor → Auto-Retrain
```

Version Control

The codebase is managed in a Git repository with two primary branches: **main** (production-ready code, triggers CD) and **develop** (integration branch, triggers CI only). Feature branches follow the pattern `feature/<ticket-id>` and are merged via pull requests with mandatory CI checks.

CI Pipeline (GitHub Actions — `ci.yml`)

- Lint: black (formatting), isort (import order), flake8 (style)
- Unit tests: pytest with coverage reporting (src/data, src/models, src/monitoring)
- Integration tests: FastAPI TestClient with mocked model
- Docker build: multi-stage build, smoke-tested with /health endpoint

CD Pipeline (GitHub Actions — `cd.yml`)

- Triggers on merge to main or manual workflow_dispatch
- Builds and pushes Docker image to GitHub Container Registry (GHCR)
- Deploys to staging server via SSH + docker compose pull && up
- Validates deployment with /health endpoint curl

5. Containerisation & Deployment

Dockerfile — Multi-Stage Build

The Dockerfile uses a two-stage build to minimise the final image size. The builder stage installs all Python dependencies into a prefix directory. The runtime stage is a slim Python 3.11 image that copies only the installed packages, application source, and configuration. The API process runs as a non-root user for security.

Docker Compose Services

Service	Image	Port	Purpose
api	fraud-detection-api	8000	FastAPI prediction server
trainer	fraud-detection-trainer	—	One-shot training (profile: training)
prometheus	prom/prometheus:2.52	9090	Metrics collection & storage
grafana	grafana/grafana:10.4	3000	Dashboards & alerting
mlflow	python:3.11-slim	5000	Experiment tracking UI

API Endpoints

Method	Endpoint	Description
GET	/health	Service health and model load status
POST	/predict	Single transaction fraud prediction
POST	/predict/batch	Batch predictions (up to 500)
GET	/model/info	Loaded model metadata and metrics
GET	/metrics	Prometheus metrics scrape endpoint
GET	/docs	Swagger UI (interactive API docs)

6. Monitoring & Maintenance

Prometheus Metrics

Metric Name	Type	Description
predictions_total	Counter	Total predictions labelled fraud / legitimate
prediction_latency_seconds	Histogram	End-to-end inference latency (p50/p95/p99)
fraud_probability_summary	Summary	Distribution of raw fraud probability scores
batch_size	Histogram	Batch prediction sizes

Drift Detection

Feature drift is assessed using the two-sample Kolmogorov-Smirnov test comparing the training reference distribution against incoming production batches. A feature is flagged as drifted when the KS p-value falls below 0.05. If more than 20% of features are flagged, or if the live F1-Score drops below 0.85, the retraining pipeline is triggered automatically.

Retraining Strategy

Trigger	Condition	Action
Scheduled	Every Sunday 02:00 UTC	Full retrain pipeline
Performance drift	Test F1 < 0.85	Retrain + redeploy
Feature drift	>20% features KS p < 0.05	Retrain + redeploy
Manual	scripts/retrain.py	On-demand retrain

7. Challenges & Solutions

Extreme Class Imbalance (0.172% fraud)

Standard accuracy metrics are meaningless. SMOTE oversampling applied exclusively to the training set prevents data leakage while dramatically improving minority class recall. PR-AUC was chosen as the primary metric rather than ROC-AUC, which can be misleading on highly imbalanced datasets. `scale_pos_weight` was included as an Optuna search parameter to provide XGBoost's native cost-sensitive learning as an alternative path.

Hyperparameter Search Efficiency

Grid Search over XGBoost's large parameter space is computationally prohibitive. Optuna's TPE sampler provides Bayesian optimisation, converging to good regions of the search space significantly faster. MedianPruner terminates unpromising trials early, further reducing total compute time.

Container Image Size

Installing the full ML stack (XGBoost, scikit-learn, pandas, etc.) produces large images. A multi-stage Docker build separates dependency installation from the runtime layer, reducing the final image by approximately 40%.

Avoiding Data Leakage in Monitoring

The drift monitor uses the training set as the reference distribution rather than the test set, ensuring the test set remains a truly held-out evaluation of generalisation performance.

Model Versioning

MLflow tracks every training run with parameters, metrics, and model artefacts. The GitHub Actions CD pipeline pins images to SHA digests, ensuring rollback capability to any previous deployment.

8. Lessons Learned

- Evaluation metric selection is as important as model selection. PR-AUC is the correct primary metric for fraud detection; ROC-AUC alone masks poor precision at the operating threshold.
- SMOTE must be applied after the train/test split to prevent synthetic samples from leaking information about the test distribution into training.
- Multi-stage Docker builds are non-negotiable for ML services: dependencies like XGBoost and pandas make single-stage images unreasonably large.
- GitHub Actions caching (type=gha for Docker layers, pip cache) reduces pipeline duration by 60–70% after the first run.
- Monitoring infrastructure should be built alongside the API, not retrofitted. Instrumenting FastAPI with prometheus-client adds negligible overhead and pays dividends immediately in production observability.
- Optuna's MedianPruner is highly effective for XGBoost HPO: it typically halves the number of full-trial evaluations needed to find a strong configuration.

9. Conclusion

This project demonstrates a complete, production-oriented MLOps pipeline for credit card fraud detection. Starting from a severely imbalanced dataset, the pipeline produces a high-quality XGBoost classifier (ROC-AUC ~0.98, F1 ~0.87) served through a documented, tested, and containerised FastAPI application. Automated CI/CD via GitHub Actions, comprehensive Prometheus/Grafana monitoring, and a data-drift-aware retraining strategy ensure the system remains reliable and maintainable as conditions evolve.

The principles applied here — reproducibility through MLflow, infrastructure-as-code through Docker Compose, safety through automated testing, and observability through metrics — generalise directly to production ML systems at scale and reflect current industry best practices as described in *Practical MLOps* (O'Reilly, 2021).

References

- Gift, N. & Deza, A. (2021). *Practical MLOps: Operationalizing Machine Learning Models*. O'Reilly Media.
- Dal Pozzolo, A. et al. (2015). Calibrating probability with undersampling for unbalanced classification. *IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*.
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD 2016*.
- Akiba, T. et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD 2019*.
- Chawla, N. V. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Kaggle. (2023). Credit Card Fraud Detection Dataset.
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>